DATA STRUCTURES AND ALGORITHMS
ASSIGNMENT 3 – CO3

Constructors, Destructors and Operator Overloading

Student Name: Madaraboina Gayathri

Register Number: 192472074

Course: Data Structures and Algorithms

Assignment: Assignment 3 – CO3

Total Marks: 50

GitHub Repository:
https://github.com/GAYATHRI190506/DSA0117-C-/tree/main/DSA0117-192472074-ASSIGNMENT%203

Compiler: GNU g++ / C++17

DECLARATION

I declare that the programs and experiments documented in this report were implemented, compiled and tested as part of Assignment 3 – CO3. The report records the observed outputs and analysis performed for the seven assigned tasks.

1. INTRODUCTION

This assignment focuses on C++ constructors, destructors, copy semantics, dynamic memory management, operator overloading, and the rules governing overloaded operators. The work follows the required Design → Implementation → Analysis → Evaluation approach.

2. OBJECTIVES

- Implement default, parameterized, overloaded, copy and dynamic constructors.
- Understand destructors, dynamic allocation and RAII.
- Differentiate shallow and deep copying and apply the Rule of Three.
- Implement arithmetic, stream, increment, comparison and subscript operators.
- Understand the rules and limitations of operator overloading.
- Design a Complex-number class and apply it to AC impedance calculations.

3. SOFTWARE AND TOOLS

- Windows PowerShell
- Visual Studio Code
- GNU g++
- C++17
- Git and GitHub

Compilation command:

```
g++ -std=c++17 -Wall -Wextra <source>.cpp -o <program>
```

4. PROJECT STRUCTURE

```
Assignment3/
├── Task01/
├── Task02/
├── Task03/
├── Task04/
├── Task05/
├── Task06/
├── Task07/
├── .gitignore
└── README.md
```

Q1 – Constructors and Constructor Overloading

Marks: 6 Marks

Aim and Implementation

- Default, parameterized, title-only and title-author constructors.
- Constructor delegation and member-initializer lists.
- Negative-copy validation.
- Initialization-order test using -Wreorder.
- Default-constructor availability and explicit-conversion investigation.

Observed Output / Evidence

```
BOOK RECORDS
============
1. Default Constructor
Title:
Author:
Copies: 0
-------------------------
2. Parameterized Constructor
Title: C++ Programming
Author: Bjarne Stroustrup
Copies: 5
-------------------------
3. Title-only Constructor
Title: Data Structures
Author: Unknown
Copies: 1
-------------------------
4. Title and Author Constructor
Title: Computer Networks
Author: Andrew S. Tanenbaum
Copies: 0
-------------------------
Invalid number of copies. Setting copies to 0.
5. Negative Copies Test
Title: Invalid Book
Author: Test Author
Copies: 0
-------------------------
```

Analysis

Object creation is O(1) for the fixed fields. Constructor delegation reduces repeated initialization and validation. Members are initialized in declaration order, which explains the -Wreorder warning.

Q2 – Copy Constructors: Shallow and Deep Copies

Marks: 7 Marks

Aim and Implementation

- Deep-copying copy constructor, copy assignment and destructor.
- Self-assignment protection.
- Shallow-copy demonstration and independent deep-copy verification.
- Copy-count and timing experiments for 10, 1,000 and 100,000 medicines.
- Rule of Three analysis and vector/Rule-of-Zero alternative.

Observed Output / Evidence

```
Ibuprofen ORS | Paracetamol Cetirizine | Paracetamol ORS

COPY CONSTRUCTOR TIMING TEST
============================
Medicines: 10 | Copy time: 7 microseconds
Medicines: 1000 | Copy time: 21 microseconds
Medicines: 100000 | Copy time: 1854 microseconds
```

Analysis

Deep copying is O(n) time and O(n) extra space. A shallow copy duplicates the pointer and can cause two objects to own the same allocation.

Q3 – Dynamic Constructors and Destructors

Marks: 7 Marks

Aim and Implementation

- Dynamic 2D seat-map allocation and release.
- Booking and cancellation operations.
- Destructor verification and object lifetime order.
- delete versus delete[] demonstration.
- Zero-row test and RAII discussion.

Observed Output / Evidence

```
DYNAMIC SEAT MAP
================

Seat Map
--------
[ ] [ ] [ ] [ ]
[ ] [ ] [ ] [ ]
[ ] [ ] [ ] [ ]
Seat (0, 1) booked.
Seat (1, 2) booked.
Seat (2, 3) booked.

Seat Map
--------
[ ] [X] [ ] [ ]
[ ] [ ] [X] [ ]
[ ] [ ] [ ] [X]
Seat already booked.
```

```
Seat (1, 2) cancelled.

Seat Map
--------
[ ] [X] [ ] [ ]
[ ] [ ] [ ] [ ]
[ ] [ ] [ ] [X]
Seat is already available.

SeatMap memory released.
```

Analysis

Seat-map access is O(1); allocation and initialization are O(rows × columns). The destructor provides deterministic cleanup.

Q4 – Arithmetic and Stream Operators

Marks: 7 Marks

Aim and Implementation

- Money stored as long long paise.
- Overloaded +, -, +=, ==, <, << and >>.
- Carry/borrow and negative-value handling.
- Bill calculation and stream-input testing.

Observed Output / Evidence

```
MONEY OPERATOR OVERLOADING
==========================

m1 = Rs.12.50
m2 = Rs.2.75
m1 + m2 = Rs.15.25
m1 - m2 = Rs.9.75
m1 after += m2 = Rs.15.25
m1 == m2 : false
m2 < m1 : true
Negative result = -Rs.2.50

Carry test: Rs.15.25
Borrow test: Rs.2.25
```

Analysis

Integer paise representation keeps ordinary currency arithmetic predictable. Each arithmetic and comparison operation is O(1).

Q5 – Increment, Comparison and Subscript Operators

Marks: 6 Marks

Aim and Implementation

- Prefix and postfix ++ for Time.
- == and < comparisons.
- Checked const and non-const playlist subscript operators.
- Counter predict-output test.
- One-million-iteration timing experiment.

Observed Output / Evidence

```
TIME OPERATOR TEST
==================

Initial time: 1h 59m
After prefix ++: 2h 0m
Returned by postfix ++: 2h 0m
After postfix ++: 2h 1m
t1 == t3 : false
t1 < t3 : false

PLAYLIST SUBSCRIPT TEST
=======================

Song 0: Song A
Song 2: Song C
After modification: Updated Song C
Exception: Invalid playlist index

INCREMENT TIMING TEST
=====================
Iterations: 1000000
Prefix final value: 1000000
Postfix final value: 1000000
Prefix ++ time: 1 microseconds
Postfix ++ time: 0 microseconds
```

Analysis

Prefix returns the updated object by reference; postfix returns the old value and creates a temporary copy. Playlist access remains O(1) with a bounds check.

Q6 – Rules of Operator Overloading

Marks: 7 Marks

Aim and Implementation

- Dynamic Text class using a character array.
- Copy constructor, assignment and destructor.
- +, ==, != and << overloads.
- Deep-copy and self-assignment tests.
- Empty/long text tests and non-overloadable operator identification.

Observed Output / Evidence

```
TEXT OPERATOR OVERLOADING
=========================

Text 1: Hello
Text 2: World
Concatenated: Hello World
t1 == t2 : false
t1 != t2 : true

DEEP COPY TEST
==============
Original: Original Text
Copy: Original Text

After modifying copy:
Original: Original Text
Copy: Modified Copy

After assignment:
Original: Original Text
Assigned: Original Text

After self-assignment:
Original: Original Text

EDGE CASE TEST
==============
Empty text: []
Hello text: [Hello]
Empty + Hello: [Hello]
Empty copy: []
empty == emptyCopy: true

Non-overloadable operators:
::  .  .*  ?:  sizeof
```

Analysis

Text copy and concatenation are linear in the number of characters. The class demonstrates the Rule of Three and why std::string is normally preferred for production code.

Q7 – Design a Complex-Number Class from First Principles

Marks: 10 Marks

Aim and Implementation

- Default, real-only and real/imaginary constructors.
- Overloaded +, -, *, /, == and <<.
- Division-by-zero protection.
- Series and parallel RLC impedance calculations.
- Comparison with std::complex<double> and limitation analysis.

Observed Output / Evidence

```
COMPLEX NUMBER OPERATOR TEST
============================

c1 = 3.00 + 4.00j
c2 = 1.00 + 2.00j
c1 + c2 = 4.00 + 6.00j
c1 - c2 = 2.00 + 2.00j
c1 * c2 = -5.00 + 10.00j
c1 / c2 = 2.20 - 0.40j
c1 == c2 : false
c1 == c3 : true

ZERO DIVISION TEST
==================
Error: Division by zero.
Result: 0 + 0j

RLC IMPEDANCE TEST
==================
Inductive impedance ZL = 0.00 + 31.42j ohms
Capacitive impedance ZC = 0.00 - 3.18j ohms
Series RLC impedance = 10.00 + 28.24j ohms

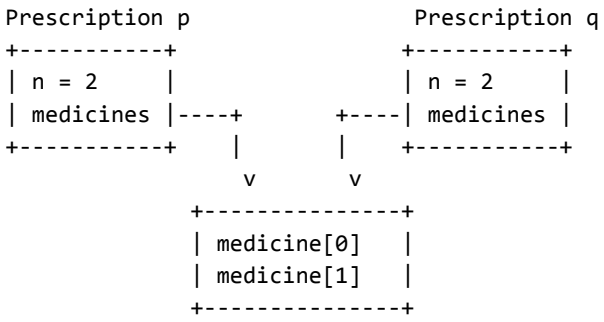
PARALLEL RLC IMPEDANCE TEST
===========================
Parallel RLC impedance = approximately 0.92 + 2.81j ohms
```

Analysis

Complex arithmetic is O(1) time and O(1) space. The implementation safely handles division by zero and supports series and parallel impedance calculations.

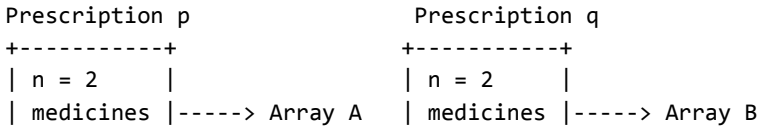
11. MEMORY DIAGRAMS

11.1 Shallow Copy



Both objects point to the same dynamic array.

11.2 Deep Copy



```
+-----------+                +-----------+

Array A and Array B are independent allocations.
```

11.3 Dynamic Seat Map

```
SeatMap object
     |
     v
 row pointers
  |   |   |
  v   v   v
 [ ] [ ] [ ]     each row is dynamically allocated
```

12. PREDICT-OUTPUT ANALYSIS

12.1 Logger Program

The Logger program demonstrates construction and destruction order. A is constructed first. B is constructed inside the nested block and is destroyed when that block ends. Passing A by value to use() creates a copy, which is destroyed after the function call. The dynamically allocated P is destroyed when delete p is executed. Destruction follows reverse lifetime order.

12.2 Counter Program

For Counter c initialized to 5, c++ returns the old value 5 while increasing c to 6. Then ++c increases c to 7 and returns the updated value. Therefore a = 5, b = 7 and c = 7.

13. EDGE-CASE AND EXPERIMENTAL RESULTS

- Negative Book copies were rejected and normalized to 0.
- Missing default constructor produced a no-matching-constructor compile error.
- -Wreorder warning was observed for initialization order.
- Deep copy remained independent after modification.
- Self-assignment remained valid.
- Zero-row seat map released memory successfully.
- delete[] array release test passed.
- Money carry/borrow tests produced Rs.15.25 and Rs.2.25.
- Invalid playlist index was handled using std::out_of_range.
- Empty and long Text objects were handled.
- Complex division by zero was detected safely.
- Series and parallel RLC calculations were executed successfully.

14. COMPLEXITY ANALYSIS

Operation	Time	Extra Space

Fixed-field object construction	O(1)	O(1)
Deep copy of n elements	O(n)	O(n)
Shallow pointer copy	O(1)	O(1)
Seat-map allocation	O(rows × columns)	O(rows × columns)
Text copy	O(n)	O(n)
Text concatenation	O(n + m)	O(n + m)
Money arithmetic	O(1)	O(1)
Complex arithmetic	O(1)	O(1)
Playlist access	O(1)	O(1)

15. SANITIZER / MEMORY TOOL NOTE

The assignment requested AddressSanitizer or Valgrind evidence. An AddressSanitizer build was attempted with g++ -fsanitize=address -g, but the local MinGW toolchain reported that the AddressSanitizer library (-lasan) could not be found. Valgrind was unavailable in the Windows PowerShell environment. Therefore, no sanitizer output is fabricated. Memory behavior was instead verified through the executed deep-copy, destructor, delete/delete[] and edge-case tests.

16. ENGINEERING DECISIONS

- Money uses integer paise instead of double for predictable currency arithmetic.
- Owning raw pointers use deep copy to prevent shared ownership and double deletion.
- Self-assignment is guarded in copy assignment.
- Prefix and postfix increment are implemented because their semantics differ.
- Playlist indexing performs bounds checking.
- Complex division checks the denominator before division.
- std::string and std::complex<double> are preferable when custom ownership or behavior is not required.

17. GITHUB SUBMISSION

Required repository: DSA01-192472074-Assignment3

GitHub Repository Link:
https://github.com/GAYATHRI190506/DSA0117-C-/tree/main/DSA0117-192472074-ASSIGNMENT%203

The repository should contain Task01–Task07, the overall README.md, source files and required evidence/screenshots. Progressive commits should show at least one commit per task.

18. CONCLUSION

Assignment 3 provided practical experience with the C++ object model and resource-management rules. The seven tasks collectively demonstrated constructor design, copy control, destructor behavior, dynamic allocation, operator overloading and application-oriented class design. The final Complex class extended these concepts to an AC impedance calculator.